

Mail-Server-Factory

Betreiben Sie Ihren Mailserver wie ein Profi - beschreiben Sie ihn in JSON und stellen Sie ihn überall bereit.

Zusammenfassung

Mail Server Factory ist ein produktionsreifes, automatisiertes Provisionierungstool für Mailserver. Der Nutzer verfasst eine einfache JSON-Konfiguration; die Factory interpretiert diese und führt alle Installationen und Initialisierungen auf dem Zielbetriebssystem durch, um einen auf Docker basierenden, lose gekoppelten Mail-Stack über 12 Verbindungstypen bereitzustellen.

Kurzbeschreibung

Ein Kotlin-/Shell-Tool, das eine JSON-Beschreibung in einen vollständig installierten, Docker-basierten Mailserver verwandelt. Es unterstützt 12 Verbindungstypen (SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt und weitere), ein umfassendes Sicherheitsframework, 25 Linux-Distributionen und liefert 439 erfolgreiche Tests mit.

Ausführliche Beschreibung

Einen echten, sicheren Mailserver aufzusetzen, ist einer der klassischen Initiationsriten in der Systemadministration – und einer der zuverlässigsten Wege, sich zu quälen. Postfix, Dovecot, TLS-Zertifikate, DNS-Einträge, Firewall-Regeln und distributionsspezifische Eigenheiten müssen perfekt aufeinander abgestimmt sein, und ein einziger falscher Befehl führt zu stillschweigend abgewiesenen Mails oder einem offenen Relay. Mail Server Factory nimmt diesen gesamten Fundus an mühsam erworbenem, fehleranfälligem Know-how und packt ihn in Software. Statt jede Komponente manuell auf einem unbekannten Betriebssystem zu konfigurieren, beschreibt der Endnutzer das gewünschte Ergebnis in einem einfachen JSON-Dokument; die Factory liest dieses JSON ein und führt die exakten Installations-

und Initialisierungsschritte auf dem Zielbetriebssystem aus, um einen Mail-Stack bereitzustellen, der auf Docker läuft – mit lose gekoppelten Komponenten, ein Designprinzip, das horizontale Skalierbarkeit gewährleistet und es ermöglicht, jede einzelne Komponente isoliert zu aktualisieren oder auszutauschen. Zudem ist das Tool bewusst *reach-agnostic*: 12 Verbindungstypen erlauben es, dass dasselbe Tool und dieselbe JSON-Konfiguration auf einem lokalen Rechner, einem Remote-Host über SSH, einer Docker- oder Kubernetes-Laufzeitumgebung, Cloud-Instanzen via AWS SSM / Azure Serial Console / GCP OS Login oder VMs über Libvirt eingesetzt werden – dieselbe deklarative Beschreibung, bereitgestellt, wohin auch immer man sie lenkt.

Das Tool unterstützt 25 Linux-Distributionen aus westlichen (Ubuntu, Debian, CentOS, Fedora, AlmaLinux, Rocky, openSUSE), russischen (ALT, Astra, ROSA) und chinesischen (openEuler, openKylin, Deepin) Familien, mit unbeaufsichtigter Installation via preseed/kickstart/cloud-init/autoyast und QEMU-basierter VM-Automatisierung für Tests. Die Enterprise-Features sind umfangreich: AES-256-GCM-Verschlüsselung, erzwungene Passwort- und SSH-Schlüsselrichtlinien, automatische Firewall-Konfiguration für Mail-Ports (25/587/465/993/995), TLS/SSL mit Zertifikatsvalidierung und HSTS, Audit-Logging und RBAC. Zu den operativen Features gehören JVM-Optimierung (G1GC), Caffeine-Caching, Connection Pooling, Prometheus-kompatible Metriken, strukturiertes Logging, Hot-Reloading der Konfiguration und Secrets-Management. Das Projekt weist 439 Tests mit 100 % Erfolgsquote und ein sauberes SonarQube-Qualitätsgate auf. Es ist das Flaggschiff der Server-Factory-Organisation.

Warum wir es entwickelt haben

Die Einrichtung eines sicheren, produktionsreifen Mailservers ist bekanntermaßen fehleranfällig und betriebssystemabhängig. Mail Server Factory bündelt dieses Fachwissen in einem deklarativen JSON-Modell sowie einer Ausführungs-Engine, sodass ein korrekter, abgesicherter und Docker-basierter Mail-Stack auf jedem unterstützten Zielsystem reproduziert werden kann – ganz ohne manuelle Schritt-für-Schritt-Arbeit.

Warum es die Spielregeln ändert

Es degradiert die Bereitstellung eines Mailservers von einer spezialisierten, tagelangen und fehleranfälligen Prozedur zu einem Akt der Konfigurationserstellung – und macht diesen Vorgang zudem portabel für 12 Verbindungstypen und 25 Linux-Distributionen, wobei von Haus aus unternehmenssichere Standardeinstellungen gelten. Das Ergebnis ist reproduzierbar

und *verifizierbar*: Dieselbe JSON-Konfiguration erzeugt jedes Mal denselben abgesicherten Stack, und die 439 bestandenen Tests sowie das saubere SonarQube-Gate des Projekts sorgen dafür, dass die ausführende Engine selbst zur Rechenschaft gezogen wird – statt sich auf ihren Ruf zu verlassen.

Was innovativ ist

- Deklarative JSON → interpretierte Installation/Initialisierung auf dem Zielbetriebssystem.
- 12 Verbindungstypen (lokal, SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt und weitere) in einem einzigen Tool.
- Unterstützung für 25 Distributionen mit unbeaufsichtigter Installation (preseed/kickstart/cloud-init/autoyast) und QEMU-Automatisierung.
- Lose gekoppelter, Docker-basierter Stack für unabhängige Skalierung/Upgrades.

Herausforderungen & Lösungen

- **Heterogenität der Betriebssysteme/Distributionen:** Gelöst durch distrospezifische Rezepte, Konfigurationen für unbeaufsichtigte Installation und QEMU-basiertes Cross-Distro-Testing.
- **Erreichbarkeit vieler Bereitstellungsziele:** Gelöst durch 12 austauschbare Verbindungstypen unter einer gemeinsamen Installations-Engine.
- **Sicherheit als Standard:** Gelöst durch AES-256-GCM, erzwungene Schlüssel-/Passwortrichtlinien, automatisierte Firewall-Regeln sowie TLS/HSTS.
- **Vertrauen in die Korrektheit:** Gelöst durch eine 439 Tests umfassende Testsuite (100 % bestanden) und ein sauberes SonarQube-Gate.

Technologie-Stack (Warum + Wie)

- **Kotlin** — Die Factory-Engine und Logik für Installationsschritte (179 KB; Kotlin 2.0.21).
- **Shell** — Bereitstellungsskripte, ISO/QEMU-Manager und OS-Automatisierung (größter Anteil in Byte).
- **Docker** — Die Laufzeitumgebung für den bereitgestellten, lose gekoppelten Mail-Stack.
- **QEMU** — VM-Automatisierung für installations- und testübergreifende Distributionen.
- **JSON** — Das benutzerorientierte, deklarative Konfigurationsformat.
- **Gradle 8.14.3 / Java 17** — Build-Toolchain.

- **Caffeine** — Mehrregionen-Caching; **G1GC-optimierte JVM** für Performance.
- **Prometheus-kompatible Metriken** — Monitoring; Grafana/ELK-kompatibel.
- **Sieve** — Mail-Filterregeln (geringer Anteil in den Sprachstatistiken).

Hinweis: GitHub kennzeichnet das Repository als Fork innerhalb der Server-Factory-Organisation. Es entstand vor der AI-Produktlinie und wird als ausgereiftes DevOps-/Bereitstellungs-Flaggschiff präsentiert, nicht als AI-Tool.